RAPORT KOŃCOWY PROJEKTU - JĘZYKI ASEMBLEROWE

Filtr medianowy dla obrazów w skali szarości

Autor: *Maciej Jazłowiecki*

Opis projektu

Celem projektu była implementacja algorytmu **Filtru Medianowego dla obrazów w skali szarości**, służącego do redukcji szumów (np. typu "sól i pieprz") na obrazach cyfrowych przy jednoczesnym zachowaniu krawędzi obiektów.

Projekt zrealizowano w architekturze hybrydowej (GUI C# + DLL C++ oraz ASM) w środowisku Microsoft Visual Studio 2022, składającej się z dwóch głównych modułów:

1. **Aplikacja Interfejsu Użytkownika (GUI):** Napisana w języku C# (Windows Forms), służąca do wczytywania obrazów, wyświetlania wyników oraz sterowania parametrami przetwarzania.
2. **Biblioteka Dynamiczna (DLL):** Napisana w C++ oraz Asemblerze x64 (MASM), odpowiedzialna za wydajne obliczenia.

Projekt realizuje kluczowe założenia:

a. Wykorzystanie instrukcji wektorowych SIMD (SSE2/SSE4) Asemblerze do równoległego przetwarzania danych (16 bajtów jednocześnie).
b. Implementuje mechanizm wielowątkowości (*multithreading*) z możliwością regulacji liczby wątków (1–12), zarządzana z poziomu C++ (dzięki wykorzystaniu biblioteki `std::thread`).
c. Implementuje algorytm przetwarzania obrazu w dwóch wariantach – dla celów porównawczych w kwestii czasu obliczeń

Język wysokiego poziomu (C++)

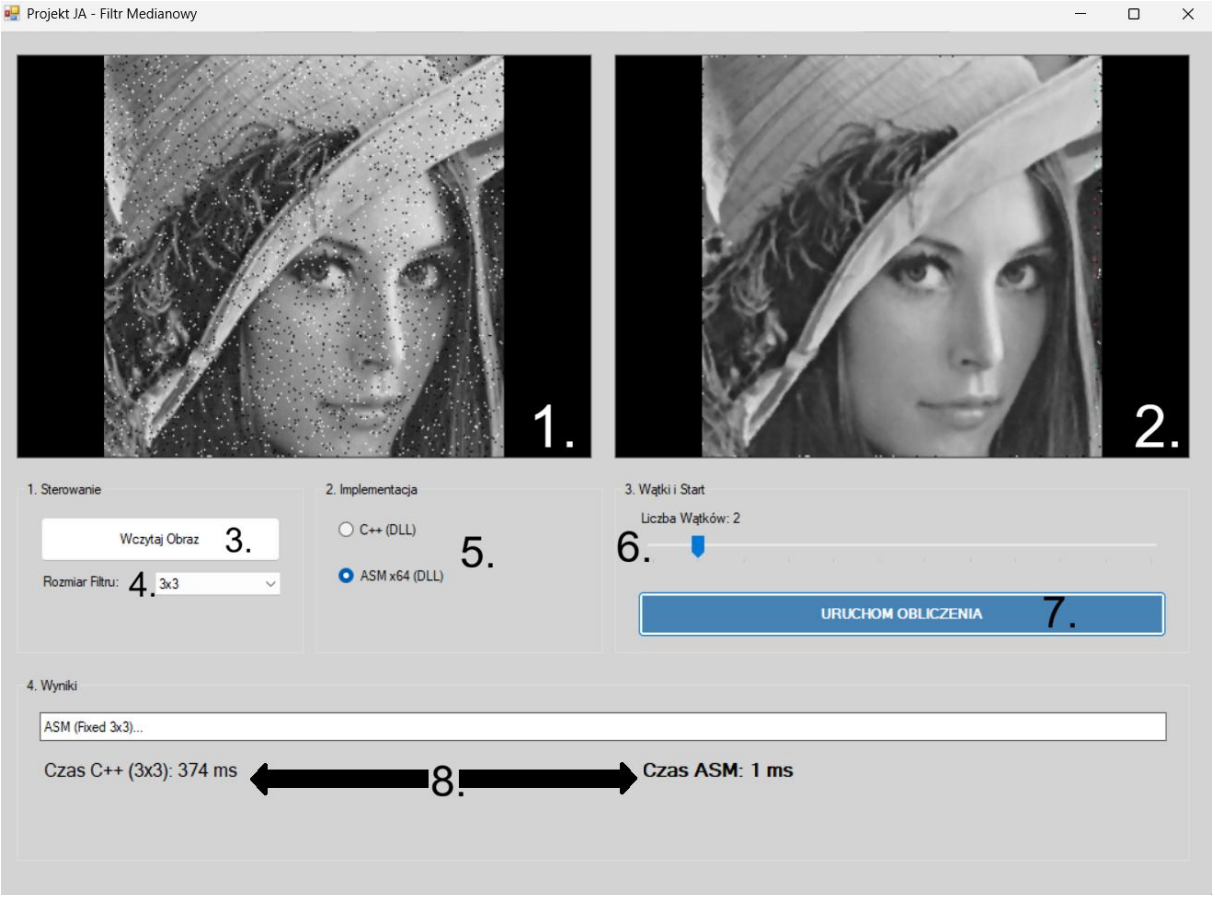
Obsługa zmiennego rozmiaru okna (3x3, 5x5, 7x7) z użyciem `std::sort`

Język niskiego poziomu (ASM x64)

Wersja algorytmu filtra dla okna 3x3 wykorzystująca makro w celu optymalizacji.

Schemat interfejsu użytkownika (UI)

Aplikacja została podzielona na cztery logiczne sekcje:



Zrzut ekranu 1. Okno (UI) Programu wraz z oznaczeniami poszczególnych elementów

I. Sekcja obrazów:

1. Lewe okno (`PictureBox`): podgląd obrazu oryginalnego
2. Prawe okno (`PictureBox`): podgląd obrazu po filtracji

II. Sekcja sterowania:

3. Przycisk „Wczytaj obraz” – otwiera okno przeglądarki plików, co pozwala użytkownikowi na wybranie konkretnego pliku graficznego celem przetworzenia
4. Lista wyboru – „**Rozmiar Filtru**” (3x3, 5x5, 7x7) – aktywna dla implementacji w języku C++

III. Sekcja wyboru implementacji:

5. Wybór trybu *C++ (DLL)* lub *ASM x64 (DLL)*

IV. Sekcja wydajności i uruchomienia:

6. Suwak (`TrackBar`) do wyboru liczby wątków *(od 1 do 12)*
7. Przycisk „**Uruchom obliczenia**” – uruchamia program wraz z wcześniej zadanymi parametrami

Opis algorytmu

Filtr medianowy jest nieliniowym filtrem cyfrowym. Dla każdego piksela obrazu pobierane jest jego otoczenie (*okno*, np. 3x3 piksele). Wartości z okna **są sortowane rosnąco**, a wartość środkowa (mediana) **zastępuje oryginalną wartość piksela**.

Wartość wyjściowa y(i,j) w punkcie obrazu jest definiowana jako mediana wartości pikseli w danym oknie W (o rozmiarze np. 3x3) wokół tego punktu

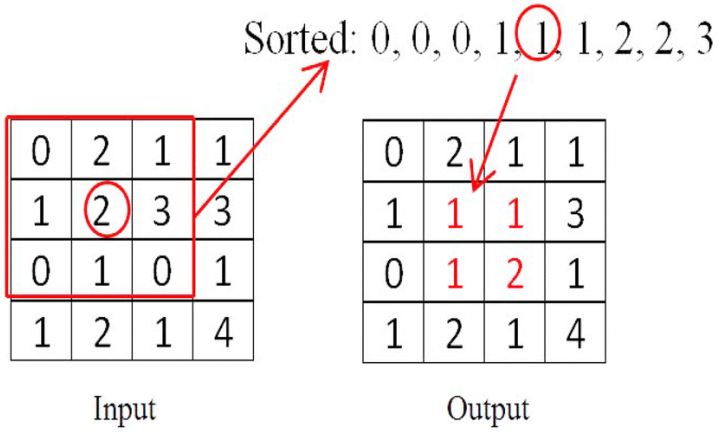
$$y(i,j) = Mediana\{x(i+k,j+l) \mid (\mathrm{k},\mathrm{l}) \in W\}$$

Gdzie:

$\mathbf{x}(i,j)$ – oryginalna wartość piksela w obrazie wejściowym

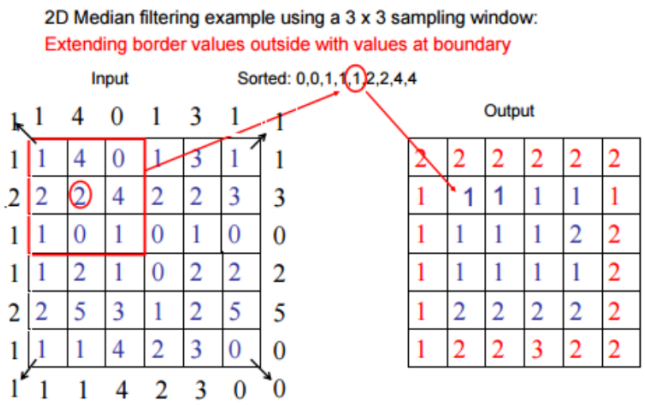
$y(i,j)$ - nowa wartość piksela w obrazie wyjściowym.

W – dwuwymiarowe okno (maska) filtru, np. 3 x 3



Grafika 2. Sortowanie elementów w tablicy pikseli (wejście/wyjście)[1]

[1] Two-step verification of brain tumor segmentation using watershed-matching algorithm - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Example-of-amedian-filter-using-a-33-window-The-pixel-values-inside-the-window-are_fig3_327021227 [accessed 26 Jan 2026]



Grafika 3. Sortowanie elementów w tablicy pikseli (non-zero padding)[2]

Implementacja C++ wykorzystuje standardowy algorytm `std::sort` (*IntroSort*). Pobiera N*N elementów do wektora, sortuje je i wybiera element środkowy.

Fragment kodu C++:

```
// standardowe sortowanie C++ (QuickSort/IntroSort)
std::sort(window.begin(), window.end());

// mediana to wartość środkowa posortowanej tablicy.
// zapisujemy ją do bufora wyjściowego (imgData).
imgData[y * stride + x * 3 + c] = window[medianIndex];
```

Implementacja ASM zamiast klasycznego sortowania, stosuje **Sieć Sortującą** opartą na instrukcjach MIN i MAX

Fragment kodu ASM:

```
SORT_MIN_MAX macro RegMin, RegMax
    movdqa xmm15, RegMin      ; zrób kopię pierwszego rejestru do pomocniczego XMM15
    pminub RegMin, RegMax     ; w RegMin zostaw tylko mniejsze wartości z pary (min)
    pmaxub RegMax, xmm15      ; w RegMax zostaw tylko większe wartości z pary (max)
endm
```

- Algorytm nie używa instrukcji warunkowych (skoków), co eliminuje potencjalne błędy predykcji rozgałęzień.
- Dzięki instrukcjom SIMD (`movdqu`, `pminub`, `pmaxub`), algorytm przetwarza **16 pikseli jednocześnie** w jednym cyklu pętli

[2] https://joiv.org/index.php/joiv/article/viewFile/151/149 [Figure 2.] [accessed 26 Jan 2026]

Opis biblioteki DLL

Biblioteka `MedianFilterLib.dll` eksportuje dwie główne funkcje zgodne z konwencją wywołań `stdcall`:

`MedianFilterCPP()`

- Zarządza podziałem obrazu na poziome pasy (chunki)
- Tworzy wątki za pomocą biblioteki `std::thread` które wywołują funkcję **`ProcessChunkCpp`**
- Każdy wątek sortuje okno o zadanym rozmiarze (3x3, 5x5 albo 7x7).

`MedianFilterASM()`

- Zarządza podziałem obrazu na pasy.
- Tworzy wątki, które wywołują procedurę asemblerową **`ProcessChunkAsm`**.
- Przekazuje do ASM dwa rodzaje parametru Stride:
  - **`srcStride`** (bezpieczny, dodatni) – do odczytu z kopii roboczej.
  - **`dstStride`** (oryginalny, może być ujemny) – do zapisu w mapie bitowej.

Zabezpieczenie przed naruszeniem pamięci - Obraz wejściowy jest kopiowany do bufora tymczasowego (`imgCopy`), aby odczyt sąsiadów odbywał się z **niezmienionych danych**.

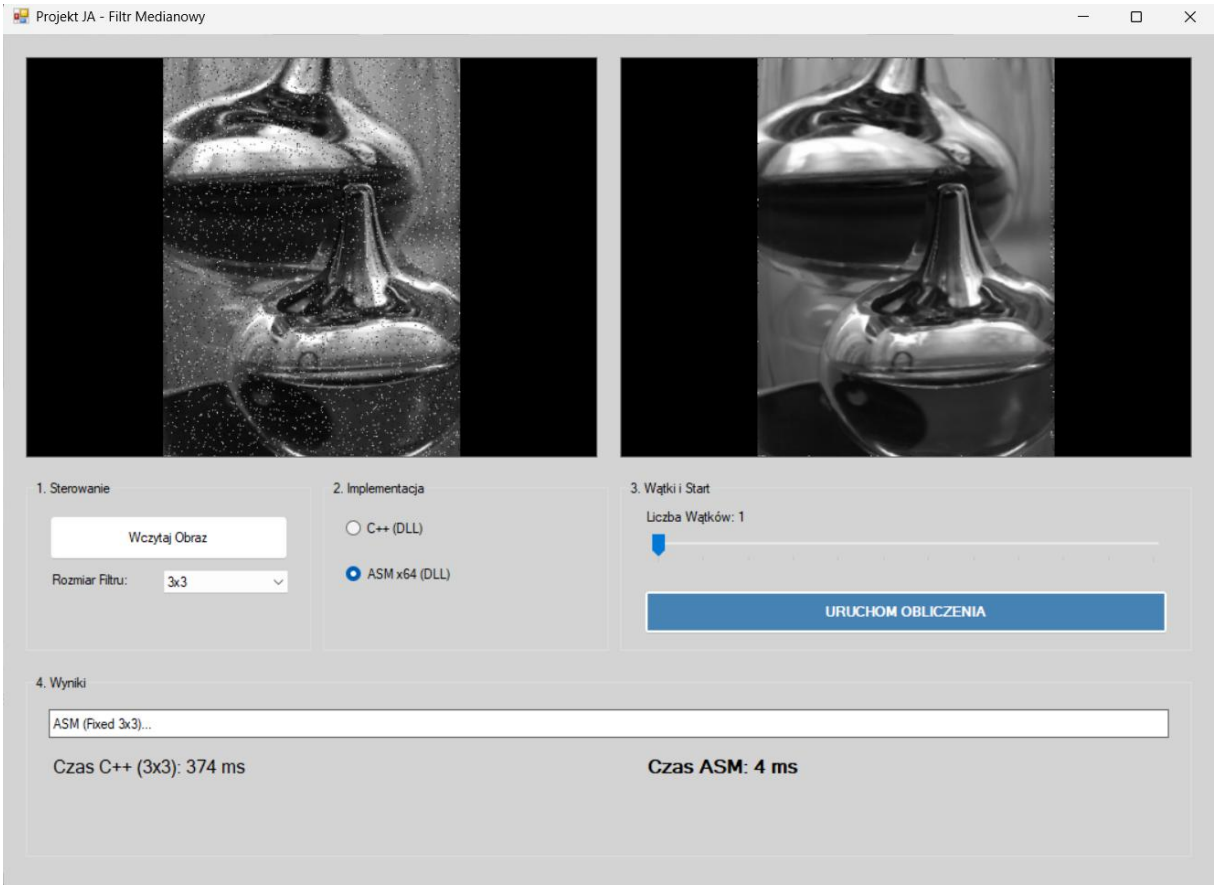
Listing wybranych fragmentów kodu ASM

Poniżej przedstawiono kluczowy fragment procedury ProcessChunkAsm odpowiedzialny za wektoryzację i sortowanie.

```
; ==============================================================================
; Makro: SORT_MIN_MAX
; bierze dwa rejestry (np. A i B). W A zostawia mniejsze wartości, w B większe.
; Działa na 16 bajtach jednocześnie (SIMD).
; ==============================================================================
SORT_MIN_MAX macro RegMin, RegMax
    movdqa xmm15, RegMin      ; zrób kopię pierwszego rejestru do pomocniczego XMM15
    pminub RegMin, RegMax     ; w RegMin zostaw tylko mniejsze wartości z pary (min)
    pmaxub RegMax, xmm15      ; w RegMax zostaw tylko większe wartości z pary (max)

endm

ProcessChunkAsm proc

    ; (...) Prolog i pobranie argumentów ze stosu
```

```
LoopX:

…

; --- Wiersz GÓRNY (y-1) ---
mov rax, rsi              ; weź adres bieżącego wiersza
sub rax, r11              ; cofnij się o jeden stride (idź wiersz wyżej)
add rax, rbx              ; przesuń się w prawo o X

movdqu xmm0, [rax - 3]    ; pobierz lewego-górnego sąsiada (i 15 kolejnych bajtów)
movdqu xmm1, [rax]        ; pobierz środkowego-górnego

movdqu xmm2, [rax + 3]    ; pobierz prawego-górnego



; (...) analogicznie dla wiersza środkowego i dolnego (XMM3-XMM8)

    ; ==========================================================
    ; SORTOWANIE
    ; używamy min/max.
    ; Celem jest, aby mediana (środkowa wartość) trafiła do XMM4.
    ; ==========================================================

    ; Faza 1: Wstępne sortowanie kolumnami
    SORT_MIN_MAX xmm0, xmm1  ; porównaj parę 0-1
    SORT_MIN_MAX xmm1, xmm2  ; porównaj parę 1-2
    SORT_MIN_MAX xmm2, xmm3  ; porównaj parę 2-3
    SORT_MIN_MAX xmm3, xmm4  ; porównaj parę 3-4
    SORT_MIN_MAX xmm4, xmm5  ; porównaj parę 4-5
    SORT_MIN_MAX xmm5, xmm6  ; porównaj parę 5-6
    SORT_MIN_MAX xmm6, xmm7  ; porównaj parę 6-7

    SORT_MIN_MAX xmm7, xmm8  ; porównaj parę 7-8 (największe wartości do xmm8)

; (...) Powtórzenie sekwencji

SORT_MIN_MAX xmm3, xmm4  ; Ostateczne ustalenie mediany w XMM4

    ; ZAPIS WYNIKU
    mov rax, rdi
    add rax, rbx
    movdqu [rax], xmm4       ; Zapis 16 przefiltrowanych pikseli

    add rbx, 16              ; Przesunięcie pętli o 16 bajtów
    jmp LoopX

    ; (...) Epilog
ProcessChunkAsm endp
```

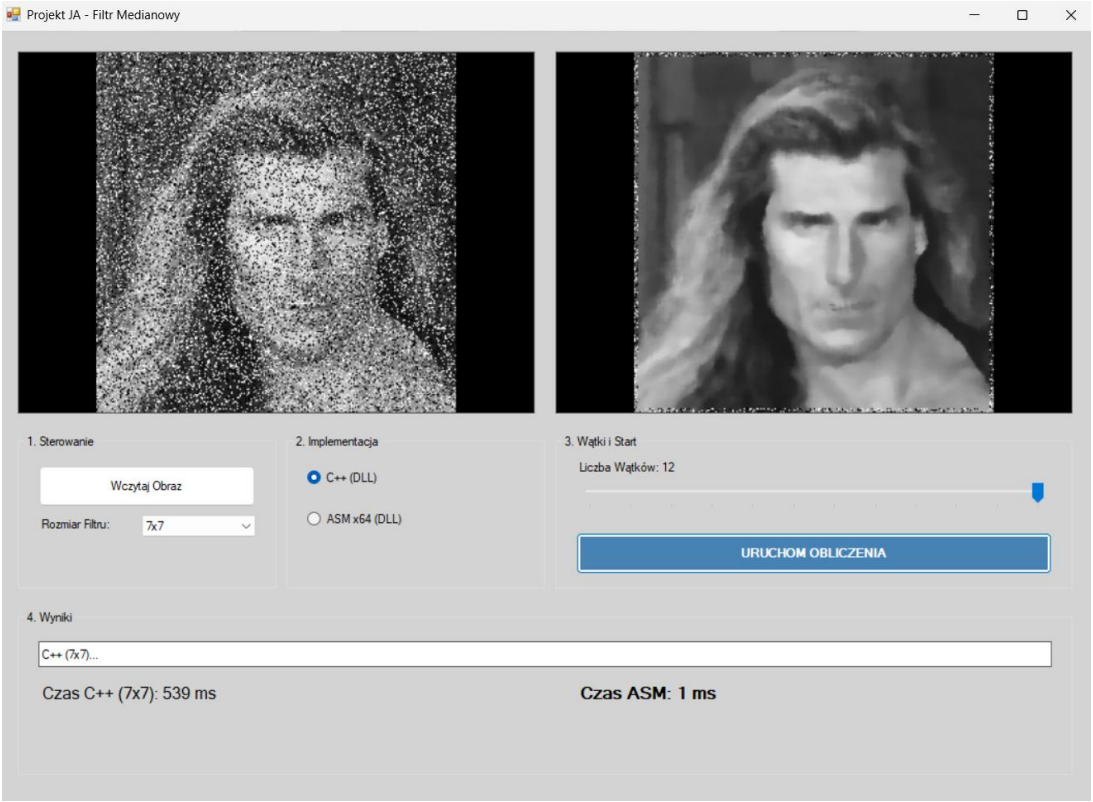
Działająca aplikacja:



Zrzut ekranu 4. Działająca aplikacja (ASM 3x3)

Na zrzucie widać poprawnie przefiltrowany obraz (zredukowany szum, lekkie rozmycie) oraz czas wykonania dla wybranej konfiguracji. (ASM okno 3x3) 1 wątek -> 4ms czas wykonania.

Na poniższym zrzucie ekranu zastosowano działanie funkcji filtra C++ wraz z maksymalnym rozmiarem okna (7x7). Widoczne bardzo duże rozmycie, a także brak zanieczyszczeń typu salt&pepper. Dodatkowo widać „nieobrobione” krawędzie obrazu – algorytm C++ oraz ASM pomijają „ramkę” okna obrazu, ponieważ skrajne piksele nie posiadają „wszystkich” sąsiadów potrzebnych do obliczenia mediany.



Zrzut ekranu 5. Działająca aplikacja (C++ 7x7) – widoczne silne rozmycie i brak krawędzi

Analiza wyników oraz pomiary

Przeprowadzono testy wydajnościowe na komputerze wyposażonym w 12 – wątkowy procesor:

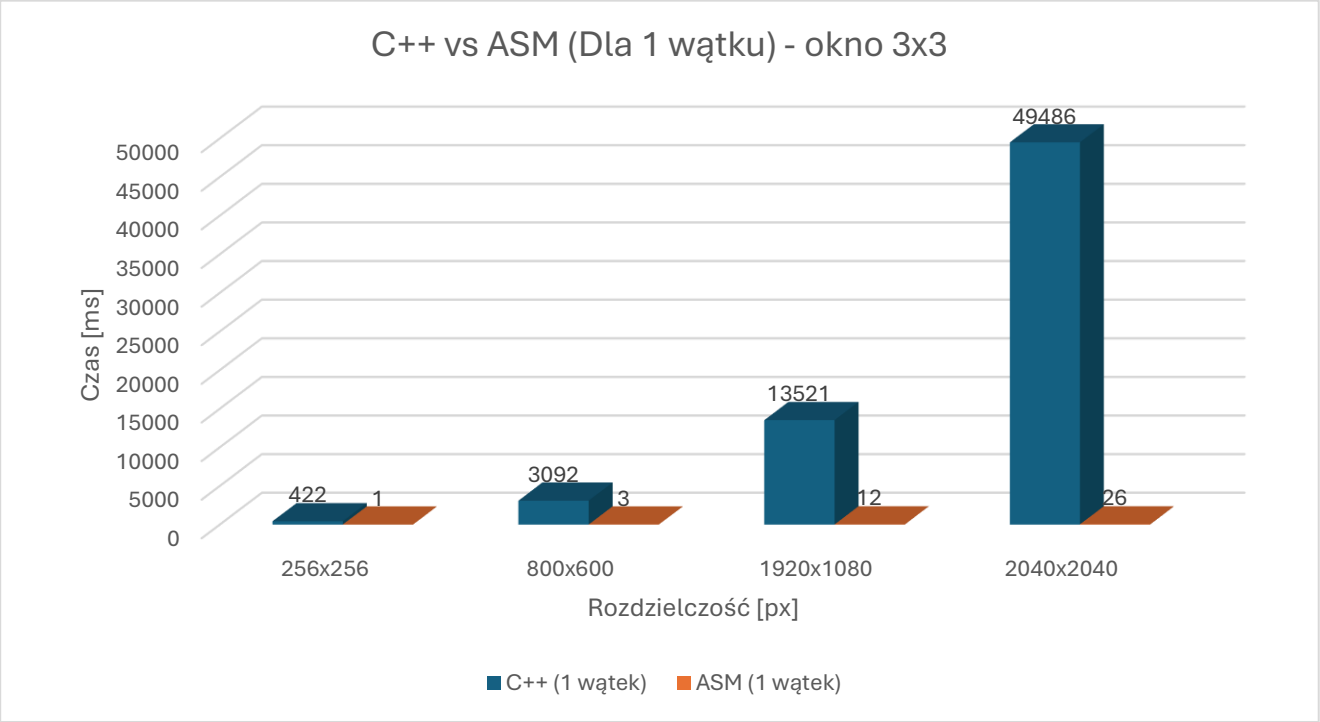
AMD Ryzen 5 5500U with Radeon Graphics (2.10 GHz)

W porównaniu wydajności wybrano 4 obazy w skali szarości - z zanieczyszczeniami typu sól i pieprz:

1. 256x256px
2. 800x600px
3. 1920x1080px (Full HD)
4. 2040x2040px (2K)

Wykres 1: **Porównanie C++ vs ASM (Dla 1 wątku) – okno 3x3**

Cel: Pokazać, ile daje sam Asembler (SIMD) bez wątków.



Wykres 2: **Skalowanie Wielowątkowości (Dla największego obrazu)**

Cel: Pokazać, jak wątki przyspieszają program (Prawo Amdahla).

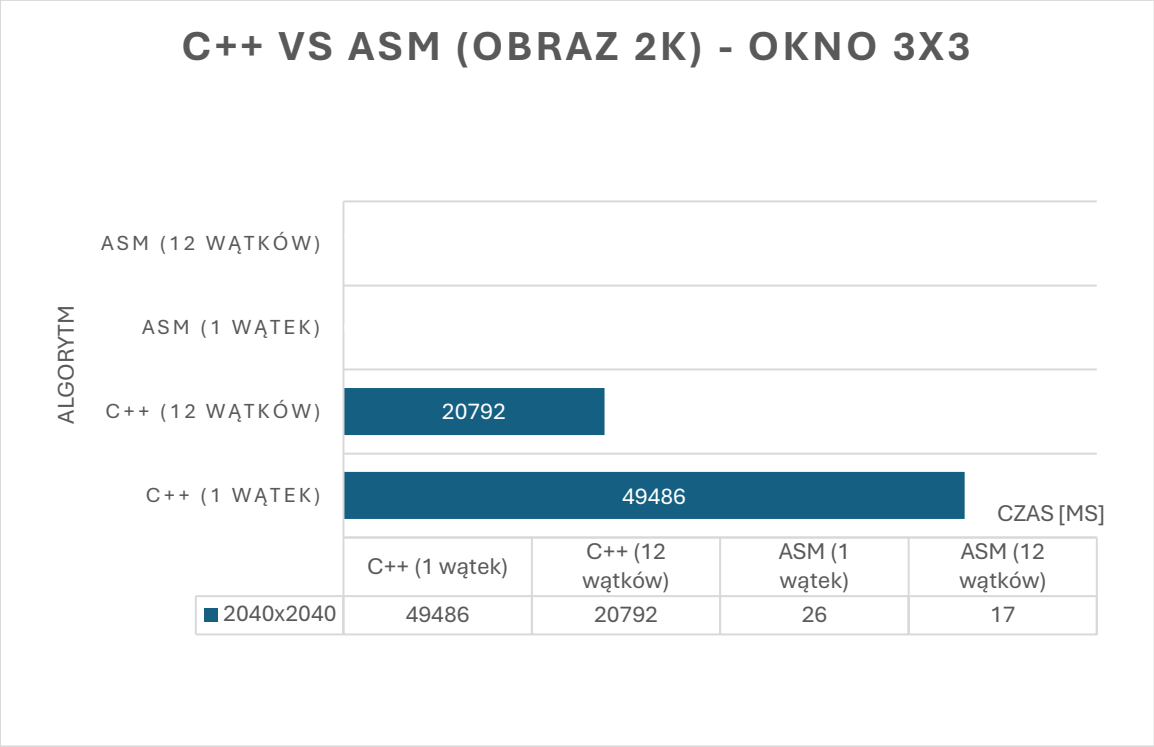


Tabela 1 – pomiary czasów dla okna 3x3 wraz z przyśpieszeniem (ASM oraz C++):

$$\text{Przyśpieszenie (speedup)} = (\text{czas C++ 1 wątek/ czas ASM 12 wątków})$$

OKNO (3X3)					
ROZDZIELCZOŚĆ	**C++ (1 wątek)**	**C++ (12 wątków)**	**ASM (1 wątek)**	**ASM (12 wątków)**	**Przyspieszenie**
256X256	422ms	773ms	1ms	4ms	105,5x
800X600	3092ms	5644ms	3ms	5ms	618,4x
1920X1080	13521ms	5092ms	12ms	10ms	1352,1x
2040X2040	49486ms	20792ms	26ms	17ms	2910,9x

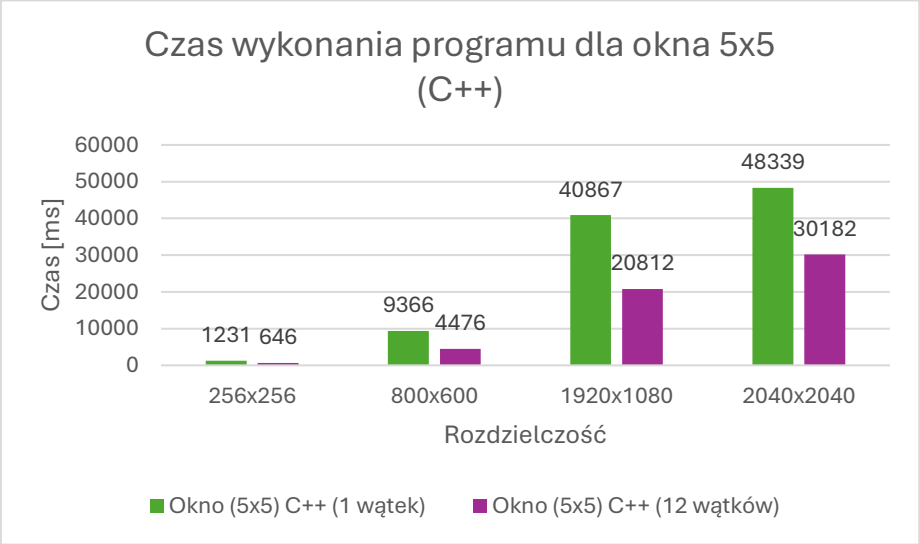
Tabela 2 – pomiary czasów dla okna 5x5 (C++):

OKNO (5X5)		
ROZDZIELCZOŚĆ	**C++ (1 wątek)**	**C++ (12 wątków)**
256X256	1231ms	646ms
800X600	9366ms	4476ms
1920X1080	40867ms	20812ms
2040X2040	48339ms	30182ms

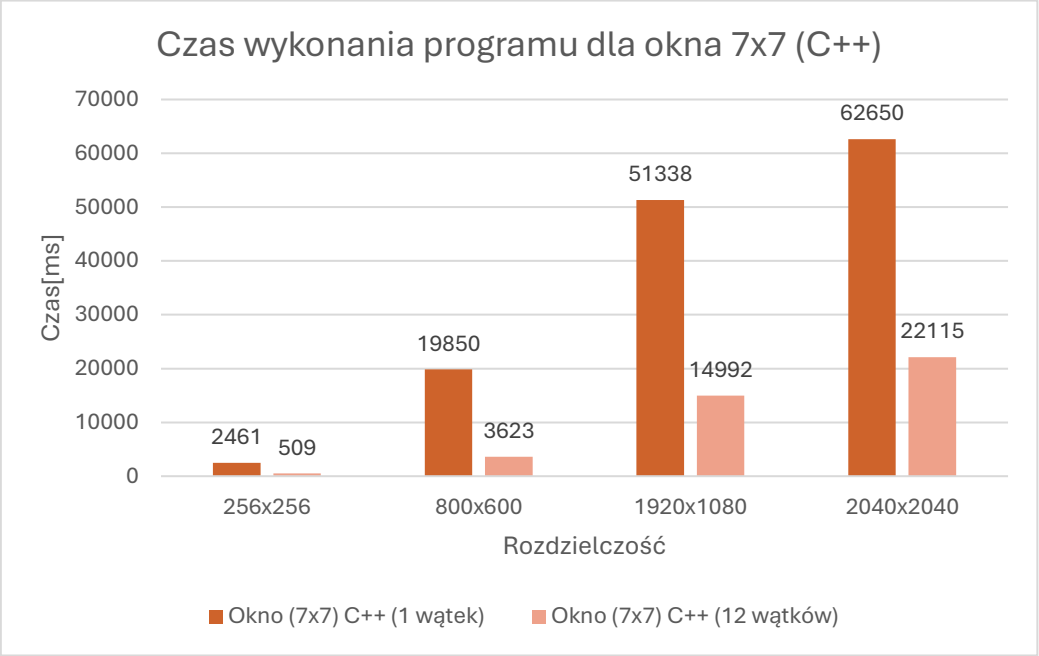
Tabela 3– pomiary czasów dla okna 7x7 (C++):

Okno (7x7)		
Rozdzielczość	**C++ (1 wątek)**	**C++ (12 wątków)**
256x256	2461ms	509ms
800x600	19850ms	3623ms
1920x1080	51338ms	14992ms
2040x2040	62650ms	22115ms

Wykres 3**. Okno 5x5 C++ - czas wykonania programu**



Wykres 4.**Okno 5x5 C++ - czas wykonania programu**



Wnioski

Na podstawie zrealizowanego projektu i przeprowadzonych pomiarów można sformułować następujące wnioski:

Dominacja optymalizacji niskopoziomowej (SIMD): Implementacja w Asemblerze wykorzystująca instrukcje wektorowe SSE (`movdqu`, `pminub`) okazała się drastycznie szybsza od referencyjnego kodu C++.

- Dla obrazu 2040x2040px czas obliczeń spadł z 49 486 ms (C++) do zaledwie 26 ms (ASM) w trybie jednowątkowym.
- Oznacza to niemal 1900-krotne przyspieszenie. Wynika to z faktu przetwarzania 16 bajtów w jednym cyklu rozkazowym oraz braku narzutu związanego z zarządzaniem kontenerami `std::vector` i generycznym algorytmem `std::sort`.

Rzeczywista efektywność wielowątkowości i "wąskie gardła": Mimo zastosowania 12 wątków, skalowanie wydajności nie jest liniowe (zgodnie z prawem Amdahla i ograniczeniami sprzętowymi).

- Dla kodu C++ (obraz 2K) czas spadł z ok. 49,5s do 20,8s, co daje przyspieszenie rzędu 2,4x (przy 12 wątkach). Sugeruje to, że wąskim gardłem stał się dostęp do pamięci RAM lub narzut na synchronizację wątków, a nie sama moc obliczeniowa CPU.
- W przypadku ASM czasy są tak krótkie (spadek z 26 ms na 17 ms), że narzut na utworzenie wątków (`std::thread`) stanowi istotną część całkowitego czasu wykonania, co powoduje zjawisko malejących korzyści.

Przewaga Sieci Sortującej nad QuickSort: Zastosowanie w Asemblerze stałej sekwencji instrukcji MIN/MAX (Sorting Network) zamiast algorytmu sortowania z instrukcjami warunkowymi wyeliminowało błędy predykcji rozgałęzień Procesor mógł w pełni wykorzystać potokowość, co przy milionach pikseli do przetworzenia dało kluczowy zysk czasowy.

Wpływ rozmiaru okna na wydajność (Złożoność obliczeniowa): Pomiary dla implementacji C++ potwierdzają wzrost złożoności obliczeniowej wraz z rozmiarem maski. Przetworzenie obrazu 2K oknem 7x7 zajęło 62 650 ms, podczas gdy oknem 3x3 trwało to 49 486 ms. Wzrost liczby elementów do posortowania (z 9 do 49) znacząco obciąża algorytm sortujący.

Krytyczna rola wyrównania pamięci (Stride): Podczas testów potwierdzono, że ignorowanie parametru Stride (szczególnie ujemnego dla bitmap "bottom-up") prowadzi do błędów naruszenia pamięci. Rozdzielenie wskaźników na źródłowy (bezpieczny) i docelowy (rzeczywisty) w kodzie ASM było niezbędne dla stabilności aplikacji.